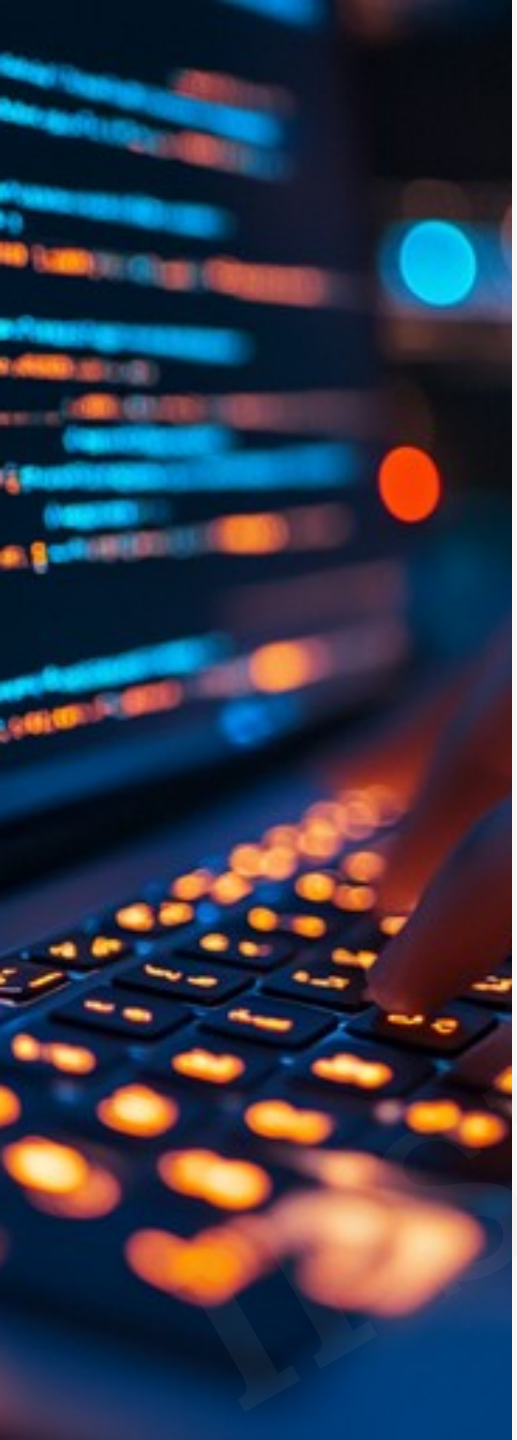


第 2 章、電腦的硬體結構

李建緯
雲科大資工系 智慧感知與系統整合實驗室
IPSI Lab, Yuntech-CSIE

IPSI Lab, Yuntech-CSIE

目錄

- CPU0 處理器
- CPU0 的指令集
- CPU0 的運作原理
- CPU0 的程式執行
- 實務案例：IA32 處理器
- 習題

IPSI Lab, yuntech-CSIE

CPU0 處理器

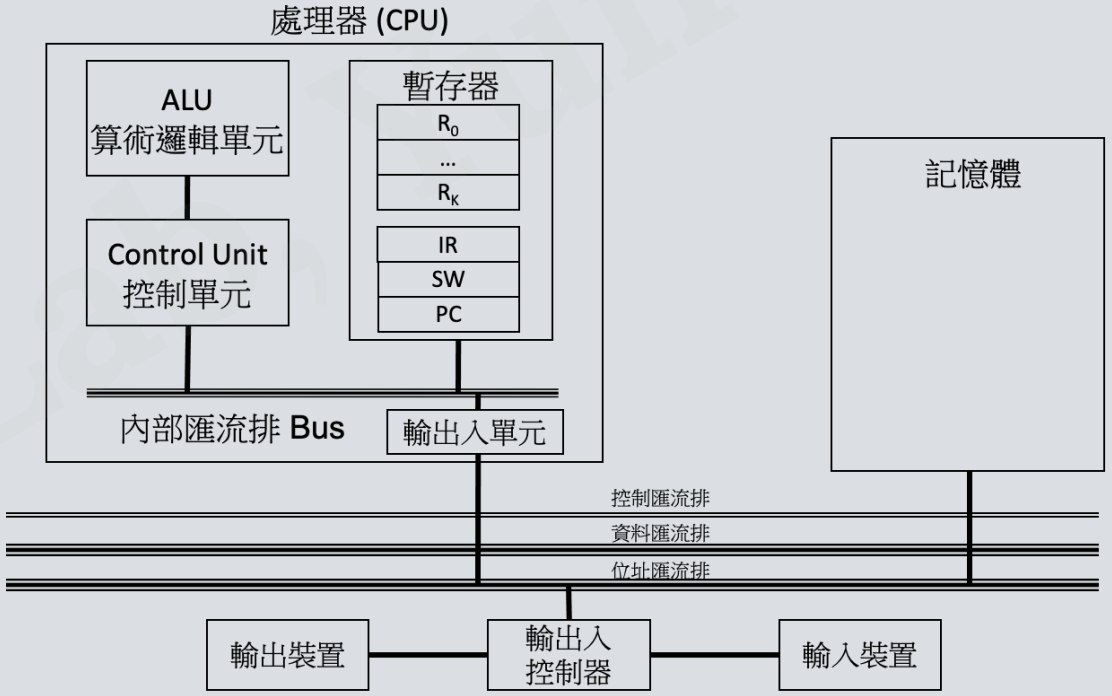
CPU0 處理器

- CPU0 是筆者所設計的一個簡易的 32 位元處理器, 主要用來說明系統程式的運作原理。
- CPU0 的設計主要是為了教學考量, 設計重點在於簡單、容易理解，因此 CPU0 幾乎不考慮成本與速度的問題。
- 商業上的處理器通常很複雜, 除了考慮成本與速度之外, 有時還會考慮相容性的問題, 因此並不容易理解。

電腦的基本結構

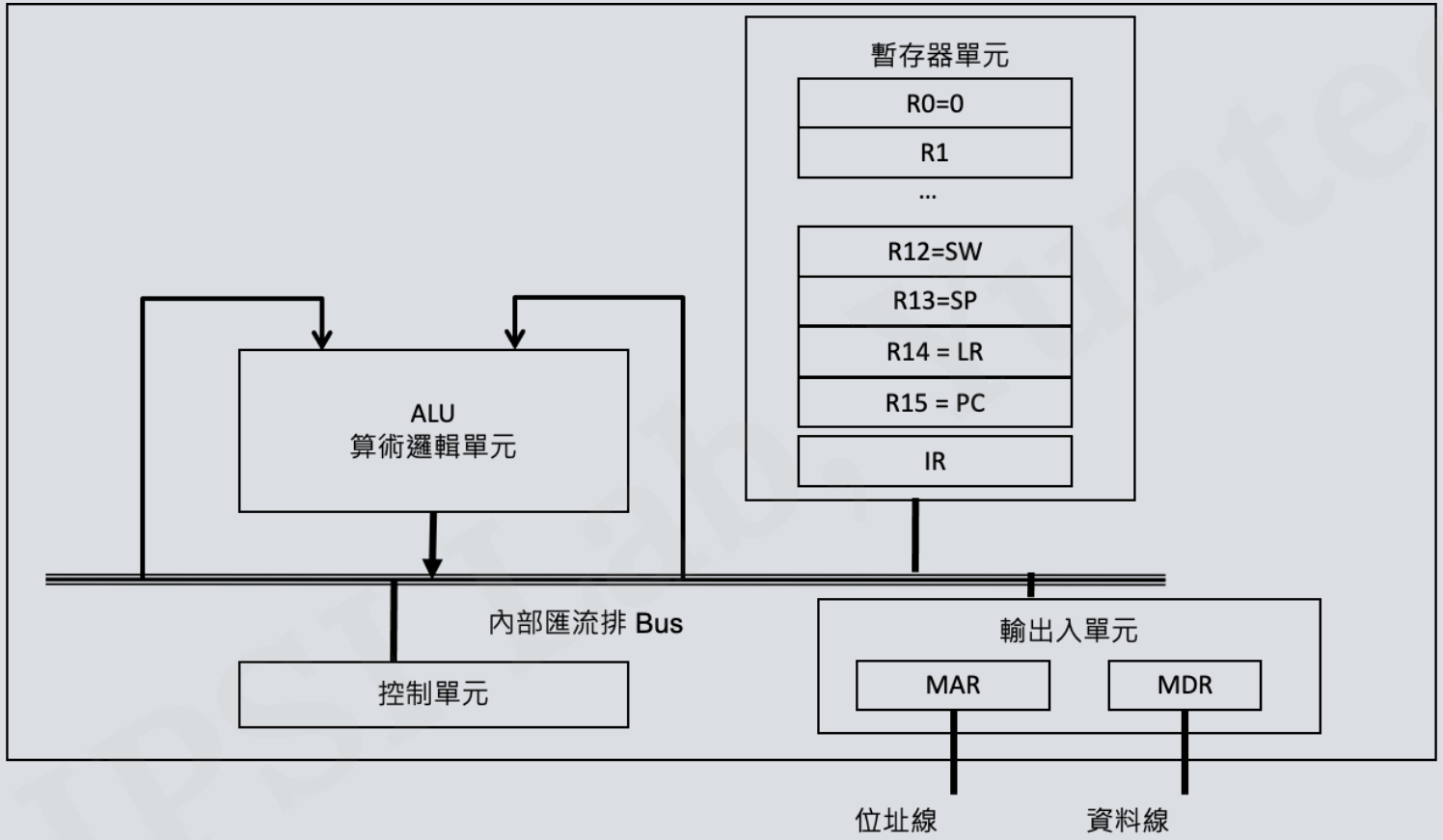
- 電腦的五大組成元件
  Computer = CPU + BUS + Memory + Input + Output

馮紐曼電腦的結構



CPU0 的結構

- **CPU0 = ALU + 暫存器（Register） + 控制單元（Control Unit）**



1. Status Word (SW)
2. Stack Pointer Register (SP)
3. Link Register (LR)
4. Program Counter (PC)
5. Instruction Register (IR)
6. Memory Address Register (MAR)
7. Memory Data Register (MDR)

IPSI Lab, Yuntech-CSIE

CPU0 的ALU（Arithmetic Logic Unit，算術邏輯單元）

- 執行**二進制數基本算術**（如加法、減法等）和**邏輯運算**（如 AND、OR、XOR 等），同時負責執行**移位**和**旋轉**等位元操作。
- ALU 可被看作是 CPU 中的「計算核心」或「小型處理器」。

IPSI Lab, Yuntech-CSIE

CPU0 的暫存器（Register）

- 主要暫存器：
  - **R0**：**唯一的唯讀暫存器**，數值固定為常數 **0**。這個暫存器常用來代表零數，並且在操作中頻繁使用。
  - **R1 - R15**：可供存取的 15 個暫存器，提供靈活的數值存取和暫存空間。

IPSI Lab, Yuntech-CSIE

- 特殊用途暫存器：
  - **R12**：**狀態暫存器（Status Word, SW）**，記錄 CPU 運行過程中的狀態資訊，例如進位、溢出等標誌位，用於後續指令的判斷。
  - **R13**：**堆疊暫存器（Stack Pointer, SP）**，指向當前堆疊頂端，用於存取函數呼叫中的局部變數和返回地址。
  - **R14**：**連結暫存器（Link Register, LR）**，保存副程式返回的位址，當副程式執行完畢時，CPU 將從該位址返回繼續執行主程式。
  - **R15**：**程式計數器（Program Counter, PC）**，保存下一條將被執行的指令的記憶體位址。

IPSI Lab, Yuntech-CSIE

CPU0 的其他組件

- **指令暫存器（IR, Instruction Register）：**
  - 儲存當前正在執行的指令，CPU 控制單元根據 IR 中的內容來控制運算的進行。
- **記憶體地址暫存器（MAR, Memory Address Register）：**
  - 用來儲存從記憶體中讀取或寫入數值的**位址**。
- **記憶體數值暫存器（MDR, Memory Data Register）：**
  - 用來儲存從記憶體中讀取的數值或待寫入的**數值**。

IPSI Lab, Yuntech-CSIE

CPU0 的控制單元（Control Unit）

- **負責解讀指令並控制其他元件的運作。根據指令暫存器（IR）中的運算碼來決定 ALU 的運算類型，並控制資料的傳輸方向。**
  - **解讀指令**：根據指令暫存器的內容控制 ALU 執行對應的運算。
  - **控制資料流的傳輸**：決定從暫存器、記憶體等元件中讀取或寫入數值。
  - **條件跳轉與分支決策**：根據狀態暫存器中的條件位來決定程式是否進行跳轉。

IPSI Lab, Yuntech-CSIE

CPU0 的執行流程

1. **取指令（Fetch）**：從記憶體中讀取指令，將其載入指令暫存器（IR）。
2. **解碼（Decode）**：控制單元根據 IR 中的運算碼來解讀指令，並設定 ALU 和其他元件的行為。
3. **執行（Execute）**：ALU 開始執行指令所要求的運算操作，並根據結果更新暫存器或狀態旗標。
4. **存回結果**：將結果寫入指定的暫存器或記憶體位置。

IPSI Lab, Yuntech-CSIE

CPU0 的指令集

CPU0 的指令集

四大類的指令

- 載入儲存 : LD, ST, ...
- 運算指令 : ADD, SUB, XOR, SHL, ROL, ...
- 跳躍指令 : JMP, JGT, JGE, ...
- 堆疊指令 : PUSH, POP, ...

IPSI Lab, Yuntech-CSIE

CPU0 的指令編碼表 – 載入儲存指令(Load / Store)

記憶體存取 (Memory Access by Immediate Offset)

- 以暫存器 + 常數 (Rb + Cx) 為位址基礎
- 適合存取陣列元素、全域變數

格式	指令	OP	說明	語法	語義	對應高階語言
L	LD	00	載入 word	LD Ra, [Rb+Cx]	Ra ← [Rb+Cx]	`Ra = mem[Rb+Cx];`
L	ST	01	儲存 word	ST Ra, [Rb+Cx]	Ra → [Rb+Cx]	`mem[Rb+Cx] = Ra;`
L	LDB	02	載入 byte	LDB Ra, [Rb+Cx]	Ra ← (byte)[Rb+Cx]	`Ra = (byte)mem[Rb+Cx];`
L	STB	03	儲存 byte	STB Ra, [Rb+Cx]	Ra → (byte)[Rb+Cx]	`mem[Rb+Cx] = (byte)Ra;`

LD: LOAD; ST: STORE; LDB: Load byte; STB: Store byte;

暫存器位址版 (Register Offset Addressing)

- 位址由兩個暫存器相加 (`Rb` + `Rc`)
- 常用於指標運算、結構體存取

格式	指令	OP	說明	語法	語義	對應高階語言
A	LDR	04	載入 word(暫存器版)	LDR Ra, [Rb+Rc]	Ra ← [Rb+Rc]	`Ra = mem[Rb+Rc];`
A	STR	05	儲存 word(暫存器版)	STR Ra, [Rb+Rc]	Ra → [Rb+Rc]	`mem[Rb+Rc] = Ra;`
A	LBR	06	載入 byte(暫存器版)	LBR Ra, [Rb+Rc]	Ra ← (byte) [Rb+Rc]	`Ra = (byte)mem[Rb+Rc];`
A	SBR	07	儲存 byte(暫存器版)	SBR Ra, [Rb+Rc]	Ra → (byte) [Rb+Rc]	`mem[Rb+Rc] = (byte)Ra;`

LDR: Load by Register

立即數載入 (Immediate Load)

- 不從記憶體取值，而是直接把常數 (Cx) 加入運算後載入
- 適合用來設定初值、常數運算

格式	指令	OP	說明	語法	語義	對應高階語言
L	LDI	08	立即載入	LDI Ra, Rb+Cx	Ra ← Rb + Cx	`Ra = Rb + Cx;`

LDI: Load immediate value;

CPU0 的指令編碼表 – 運算指令

運算指令 (資料傳送與比較)

格式	指令	OP	說明	語法	語義	對應高階語言
A	CMP	10	比較	CMP Ra, Rb	SW ← Ra >=< Rb	`if (Ra == Rb)` / 設定旗標
A	MOV	12	移動	MOV Ra, Rb	Ra ← Rb	`Ra = Rb;`

CMP: Compare; MOV: Move;

運算指令 (算術運算)

格式	指令	OP	說明	語法	語義	對應高階語言
A	ADD	13	加法	ADD Ra, Rb, Rc	Ra ← Rb + Rc	`Ra = Rb + Rc;`
A	SUB	14	減法	SUB Ra, Rb, Rc	Ra ← Rb - Rc	`Ra = Rb – Rc;`
A	MUL	15	乘法	MUL Ra, Rb, Rc	Ra ← Rb * Rc	`Ra = Rb * Rc;`
A	DIV	16	除法	DIV Ra, Rb, Rc	Ra ← Rb / Rc	`Ra = Rb / Rc;`

SUB: Subtract; MUL: Multiply; DIV; Divide;

運算指令 (邏輯運算)

格式	指令	OP	說明	語法	語義	對應高階語言
A	AND	18	AND	AND Ra, Rb, Rc	Ra ← Rb and Rc	`Ra = Rb & Rc;`
A	OR	19	OR	OR Ra, Rb, Rc	Ra ← Rb or Rc	`Ra = Rb | Rc;`
A	XOR	1A	XOR	XOR Ra, Rb, Rc	Ra ← Rb xor Rc	`Ra = Rb ^ Rc;`

IPSI Lab Yuntech CSIE

運算指令 (位元操作)

格式	指令	OP	說明	語法	語義	對應高階語言
A	ROL	1C	向左旋轉	ROL Ra, Rb, Cx	Ra ← Rb rol Cx	`Ra = rol(Rb, Cx);`
A	ROR	1D	向右旋轉	ROR Ra, Rb, Cx	Ra ← Rb ror Cx	`Ra = ror(Rb, Cx);`
A	SHL	1E	向左移位	SHL Ra, Rb, Cx	Ra ← Rb << Cx	`Ra = Rb << Cx;`
A	SHR	1F	向右移位	SHR Ra, Rb, Cx	Ra ← Rb >> Cx	`Ra = Rb >> Cx;`

IPSI Lab, Yuntech-CSIE

CMP: ROL: Rotate Left; ROR: Rotate Left; SHL: Shift Left; SHR: Shift Right

CPU0 的指令編碼表 – 跳躍指令

條件跳躍指令 (Conditional Jumps)

格式	指令	OP	說明	語法	語義	對應高階語言
J	JEQ	20	跳躍(=)	JEQ Cx	if SW(=) PC←PC+Cx	if (a==b)
J	JNE	21	跳躍(!=)	JNE Cx	if SW(!=) PC←PC+Cx	if (a!=b)
J	JLT	22	跳躍(<)	JLT Cx	if SW(<) PC←PC+Cx	if (a<b)
J	JGT	23	跳躍(>)	JGT Cx	if SW(>) PC←PC+Cx	if (a>b)
J	JLE	24	跳躍(<=)	JLE Cx	if SW(<=) PC←PC+Cx	if (a<=b)
J	JGE	25	跳躍(>=)	JGE Cx	if SW(>=) PC←PC+Cx	if (a>=b)

JEQ: Jump if Equal; JNE: Jump if Not Equal; JLT: Jump if Less Than;
JGT: Jump if Greater Than; JLE: Jump if Less or Equal; JGE: Jump if Greater or Equal

無條件跳躍 (Unconditional Jump)

格式	指令	OP	說明	語法	語義	對應高階語言
J	JMP	26	無條件跳躍	JMP Cx	PC←PC+Cx	goto label

IPSI Lab, Yuntech-CSIE

副程式與中斷 (Subroutine & Interrupt)

格式	指令	OP	說明	語法	語義	對應高階語言
J	SWI	2A	軟體中斷	SWI Cx	LR←PC; PC←Cx	系統呼叫
J	CALL	2B	呼叫副程式	CALL Cx	LR←PC; PC←PC+Cx	function()
J	RET	2C	返回	RET	PC←LR	return

SWI: SoftWare interrupt; CALL: Call Subroutine; RET: return

CPU0 的指令編碼表 – 堆疊指令

格式	指令	OP	說明	語法	語義	對應高階語言
A	PUSH	30	推入 word	PUSH Ra	SP-=4; [SP] = Ra;	`stack.push(Ra);`
A	POP	31	彈出 word	POP Ra	Ra = [SP]; SP+=4;	`Ra = stack.pop();`
A	PUSHB	32	推入 byte	PUSHB Ra	SP--; [SP] = Ra; (byte)	`stack.push((byte)Ra);`
A	POPB	33	彈出 byte	POPB Ra	Ra = [SP]; SP++; (byte)	`Ra = (byte)stack.pop();`

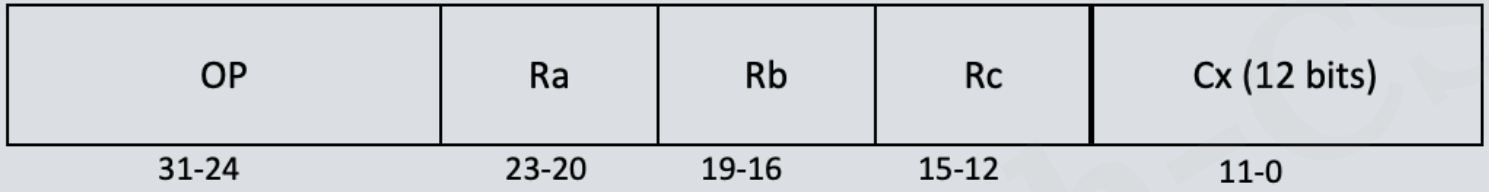
IPSI Lab, Yuntech-CSIE

CPU0 的指令格式

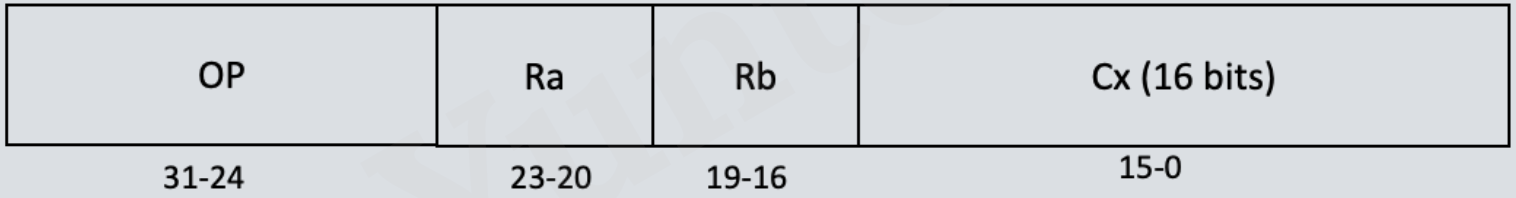
- CPU0 的指令採用 **32 位元** 寬度，三種主要格式：
  - **A 型**：**算術與邏輯運算指令（Arithmetic）**，主要用於執行算術和邏輯運算。
  - **L 型**：**載入與儲存指令（Load & Store）**，負責將數值從記憶體載入暫存器或將數值從暫存器儲存至記憶體。
  - **J 型**：**跳躍指令（Jumping）**，控制程式的執行流程，實現條件或無條件跳躍。

IPSI Lab, Yuntech-CSIE

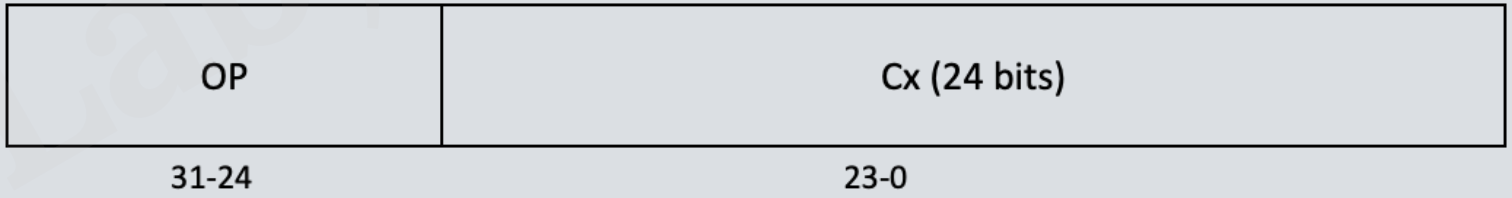
A 型



L 型



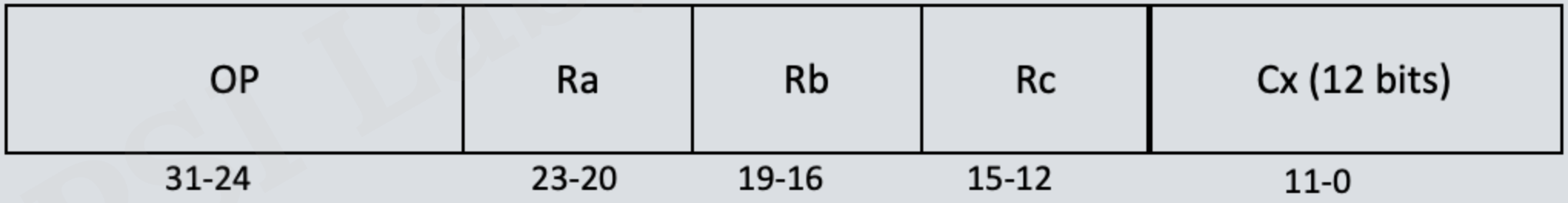
J 型



IPSI Lab, Yuntech-CSIE

A 型指令格式 (Arithmetic) 的結構

- **OP**（操作碼）：控制**指令的類型**，如加法、減法、與非等運算。位元範圍為 31-24。
- **Ra, Rb, Rc**（暫存器）：分別代表兩個操作數和一個結果暫存器。Ra 用於儲存最終的計算結果，而 Rb 和 Rc 是來源操作數。這些暫存器的位元範圍分別是：
  - Ra（23-20 位元）, Rb（19-16 位元）, Rc（15-12 位元）
  - **大小都為4位元**，恰可寫成**16進位字元**
- **Cx**（常數）：一個 12 位元的常數，可以用於立即值運算（如與某個固定數字進行加法或邏輯運算）。(2補數的表達法)

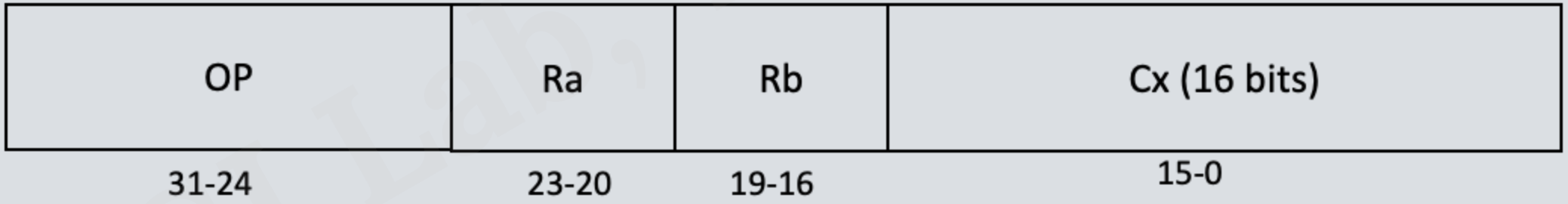


- **指令範例**
  - `ADD Ra, Rb, Rc` 將 Rb 和 Rc 的值相加，結果儲存於 Ra 中。這些指令可以在算術運算、位元操作中應用廣泛。
- **指令編碼範例：**
  - 假設 `ADD R1, R2, R3` ，其操作碼假設為 `1A` ：
    - OP = `1A` ，Ra = `R1` ，Rb = `R2` ，Rc = `R3`
    - 編碼結果：
      - `1A 1 2 3`
      - `00011010 0001 0010 0011`

IPSI Lab, Yuntech-CSIE

L 型指令格式 (Load & Store) 的結構

- **OP**（操作碼）：控制指令的類型，如載入或儲存操作。位元範圍為 31-24。
- **Ra, Rb**（暫存器）：Ra 用於儲存記憶體中的數值，Rb 用於儲存源或目標的記憶體地址。Ra 和 Rb 分別佔據 23-20 和 19-16 位元。
- **Cx**（常數）：16 位元的常數，作為**偏移量**，通常用於**基址加偏移**的模式來訪問記憶體中的數值。(2補數的表達法)



- **指令範例**
  - `LDR Ra, [Rb + Cx]` 將記憶體位址 `Rb + Cx` 的數值載入到 Ra 中。
  - L 型指令主要用於數值搬移，如從記憶體中載入數值到暫存器，或將暫存器中的數值儲存到記憶體。
- **指令編碼範例**
  - 假設 `LDR R1, [R2 + 4]`，其操作碼假設為 `24`：
    - OP = `24`，Ra = `R1`，Rb = `R2`，Cx = `0004`
    - 編碼結果：
      - `24 1 2 0004`
      - `00100100 0001 0010 0000 0000 0100`

J 型指令格式 (Jumping) 的結構

- **OP（操作碼）：控制跳躍的類型，決定是無條件跳躍還是條件跳躍，位元範圍為 31-24。**
- **Cx（常數）：一個 24 位元的位址偏移量，用於指示跳躍目的地的位址。範圍可達 ±8MB 的記憶體位址。(2補數的表達法)**
- **跳躍範圍：**
  - Cx 可跳躍的範圍為 $\pm 2^{23}$（即 ±8MB），若需要跳躍更遠的地方，可結合載入儲存指令將目標地址直接載入程式計數器。



- **指令範例**
  - `JMP 100`，這是一個無條件跳躍指令，將程式計數器（PC）設定為 `PC + 100`，使程式執行跳躍到偏移 100 的地方。
- **指令編碼範例**
  - 假設 `JMP 100`，其操作碼假設為 `3F`：
    - OP = `3F`，Cx = `000064`，（註: 100 的十六進位表示為 000064）
    - 編碼結果：
      - `3F 00 00 64`
      - `00111111 0000 0000 0000 0000 0110 0100`

IPSI Lab, Yuntech-CSIE

指令格式總結

- **A 型指令**：用於處理 CPU 的算術與邏輯運算，如加法、減法、位元運算等。適合運算密集型操作。
- **L 型指令**：負責**數值搬移**，將數值載入至暫存器或儲存至記憶體。適合資料處理與搬移。
  - 有幾個例外，分別是**LDR、STR、LBR、SBR** 等四個指令。由於這四個指令都具有三個暫存器參數，因此只能編碼為**A型格式**。
- **J 型指令**：控制程式執行流程的跳躍指令，適合流程控制與條件跳轉。

IPSI Lab, Yuntech-CSIE

編碼總結

- 各類型指令均採用 32 位元，OP 用來決定指令類型，而暫存器（Ra, Rb, Rc）和常數位元（Cx）用來儲存操作數、記憶體位址或位址偏移。

IPSI Lab, Yuntech-CSIE